Laboratorul 2 Cerințe colecții și GitHub

PCLP • An universitar 2026–2027 • Suport pentru studenți

Ideea se transformă într-un proiect realizabil. La final sunt stabilite tema, trei funcționalități de bază, o structură de date și un depozit GitHub accesibil tuturor membrilor. Este necesară explicarea contribuției individuale.

Pregătire și desfășurare

Sunt necesare configurarea de la L1, un cont GitHub și Git instalat pe calculator. Buget: 10 minute recapitulare; 20 minute liste și dicționare; 20 minute cerințe; 25 minute GitHub; 20 minute lucru la proiect; 5 minute verificare.

Lista păstrează o succesiune de elemente. Dicționarul asociază chei cu valori. O listă de dicționare poate reprezenta sarcini, cărți sau evenimente. Exemplul se scrie în `sarcini.py`:

```
sarcini = [
    {"id": 1, "titlu": "Alegerea temei", "gata": False},
    {"id": 2, "titlu": "Citirea cerințelor", "gata": True},
]
for sarcina in sarcini:
    if not sarcina["gata"]:
        print(sarcina["id"], sarcina["titlu"])
```

Se afișează doar sarcina cu identificatorul 1. Un identificator stabil este util deoarece poziția din listă se poate schimba după ștergere sau sortare.

Definirea proiectului

Se completează fișa echipei: problema rezolvată, utilizatorul, intrările, rezultatele, cele trei funcționalități obligatorii și limitele versiunii minime. Pentru fiecare funcționalitate se scrie o probă observabilă. Exemplu: „O sarcină cu titlu valid apare în listă după adăugare; un titlu gol produce un mesaj, fără adăugare”. Se schițează pe hârtie ecranul principal. Pentru traseul comun se alege Tkinter; o interfață web sau Qt se discută înainte de schimbarea tehnologiei.

Primul depozit GitHub

Git păstrează istoricul local. GitHub găzduiește depozitul și oferă colaborare. Un commit este un punct identificabil în istoric, iar push trimite commiturile către depozitul online.

1. Se creează pe GitHub un depozit privat, inițializat cu `.gitignore` pentru Python. Se invită ceilalți membri și cadrul didactic conform contului comunicat la laborator. Datele de contact nu se publică în Documentatie_proiect.docx.

2. Depozitul se clonează individual prin funcția Clone Repository din PyCharm sau prin `git clone URL`, cu înlocuirea URL prin adresa reală. Autentificarea se face prin mecanismul GitHub oferit de client; tokenurile nu se scriu în cod.

3. Se creează un mediu .venv propriu în fiecare copie locală. Se completează `.gitignore` cu `.venv/`, `.idea/`, `__pycache__/`, `.env` și `*.pyc`. Mediile virtuale nu se încarcă în GitHub.

4. Identitatea Git se configurează pentru acest depozit cu datele titularului contului. Se poate utiliza adresa de confidențialitate furnizată de GitHub.

Înainte de comenzi se copiază modelul Documentatie_proiect.docx din 00_Organizare/student în depozit și se completează problema, echipa și cerințele. Documentul se editează pe rând; fișierele Word nu se îmbină ca sursele Python.

```
git config user.name "Numele propriu"
git config user.email "adresa proprie asociata contului"
git switch -c documentare-proiect
git status
git add Documentatie_proiect.docx .gitignore
git commit -m "Documentez cerintele proiectului"
git push -u origin documentare-proiect
```

Înainte de git add se verifică fișierele listate de git status. În GitHub se deschide un pull request, adică o propunere de integrare a schimbării. Documentul Word se descarcă pentru verificare de către un coleg, apoi se integrează modificarea. Pentru sursele Python, diferențele se citesc direct în pull request. Versiunea comună se preia astfel:

```
git switch main
git pull --ff-only
```

Comenzile presupun că ramura principală se numește `main`. Modificările locale neînregistrate se salvează într-un commit pe ramura de lucru înainte de schimbarea ramurii. La conflict se oprește integrarea și se rezolvă împreună liniile semnalate.

Exerciții și predare

Se adaugă o a treia sarcină în exemplu, se marchează sarcina 1 drept finalizată și se numără sarcinile rămase. Apoi se definesc structura de date pentru proiect și o operație simplă asupra ei.

Până la L3 se predau fișa echipei, linkul depozitului și o primă funcție. Contribuția individuală se evidențiază printr-un commit explicabil sau printr-o contribuție în pereche documentată. Efort estimat: 1 –2 ore individual. La verificare se prezintă un pull request și se explică diferența dintre lista locală din memorie și istoricul Git.

Resursă: https://docs.github.com/en/get-started/using-github/hello-world